

Engineering Begins Before the First Line of Code

(9th January, 2026)

Most people imagine the role of a software engineer as simply coding a solution to a problem. But in reality, coding is just a fraction of what we do. **Six months ago, a question kept resurfacing in my mind: "If AI is getting better at writing code, what becomes the engineer's real responsibility?"**

Artificial intelligence, more specifically generative and agentic AI, has rapidly transformed the software industry. Tools such as Claude, GPT, and Cursor have become part of developers' daily workflows. (DeBellis et al., 2025) AI has evolved at a remarkable pace, from the deep learning breakthrough of AlexNet in 2012 (Krizhevsky et al., 2012) to today's multimodal, agentic systems. The scale and speed of this progress are difficult to fully grasp, especially for students who are still building their technical foundations. Like many others, I questioned whether the role I saw myself pursuing would continue to exist. If AI could increasingly handle implementation, then where would software engineers continue to create value?

The answer became clearer during Arrayah, a startup accelerator program for bullish builders. My co-founder and I entered with what we believed was a strong idea: an automated compliance solution for architects. Initially, we believed the challenge was extracting compliance rules from lengthy, unstructured planning documents. However, after speaking with architects, we realised that extracting the rules was only one part of the problem. The real challenge was helping architects interpret those rules, identify opportunities and permissible design variations within them, and use that understanding to create smarter design plans. Those conversations challenged our assumptions and reshaped our understanding of the problem, ultimately changing the direction of our product. Rather than asking, *"How do we build this?"* we found ourselves repeatedly asking, *"Have we correctly understood the problem we're trying to solve and how do we best solve it?"* It's the difference between treating a symptom and diagnosing the disease. One provides temporary relief, the other solves the underlying problem. That shift fundamentally changed how I viewed software engineering.

Software engineering isn't just about writing code. It's about understanding problems well enough to know what should be built, why it should be built, and whether the solution actually creates value. This philosophy isn't unique to my experience. Amazon's *Working Backwards* approach begins by defining the customer problem and desired outcome before implementation. (Bryar & Carr, 2021) The principle is simple: don't start building until you understand what your customers actually need. Similarly, Agile Scrum encourages teams to build incrementally, gather feedback, and continuously refine their solutions. It recognises that understanding evolves, and therefore software must evolve with it. (Schwaber & Sutherland, 2020)

This made me realise something that AI has unintentionally highlighted. For years, many people, including myself, associated software engineering primarily with writing code. Today, AI

is becoming increasingly capable of generating that code. But AI cannot determine whether we have correctly understood the problem. It cannot independently uncover user needs, challenge assumptions, balance technical and business trade-offs, or decide whether a solution creates genuine value. Those responsibilities still belong to engineers.

Rather than changing software engineering, AI is exposing what software engineering has always been. Coding has never been the end goal; it has always been the tool. The real value of a software engineer lies in asking better questions, understanding users, exercising sound judgement, and making decisions that technology alone cannot. AI will continue to make implementation faster, but I believe engineering judgement, the ability to determine what should be built, why it matters, and how it creates value, will continue to distinguish exceptional software engineers in the years ahead.

Bryar, C., & Carr, B. (2021). *Working Backwards: Insights, Stories, and Secrets from Inside Amazon*. St. Martin's Press.

DeBellis, D., Storer, K., Harvey, N., et al. (2025). *DORA 2025 State of AI-assisted Software Development Report*. Google Cloud DORA.

Krizhevsky, A., Sutskever, I., & Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. *Advances in Neural Information Processing Systems*, 25, 1097-1105.

Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide*.